

AUTD3 触覚ボードキット
サンプル解説
Python 編

版	日付	内容
1.0	2026/03/19	初版

目次

1. 概要	2
2. ハードウェア接続	2
3. 起動方法	3
4. サンプルプログラム実行	4
4.1. サンプル一覧	4
4.2. 実行方法	5
5. サンプルプログラム解説	6
5.1. 共通	6
5.2. 単一焦点サンプル(sample01_single.py)	8
5.3. 複数焦点サンプル(sample02_multi.py)	14
5.4. 焦点移動及び ON/OFF サンプル(sample03_move.py)	17
5.5. ベッセルビームサンプル(sample04_bessel.py)	21
5.6. STM サンプル(sample05_stm.py)	25
6. その他	28
6.1. 独自プログラムの作成と実行	28
6.2. 実行時のエラーなど	29

1. 概要

本書は、AUTD3 触覚ボードキットに付属する Python 版サンプルについて、実行方法、及び、コードの解説等を記載いたします。

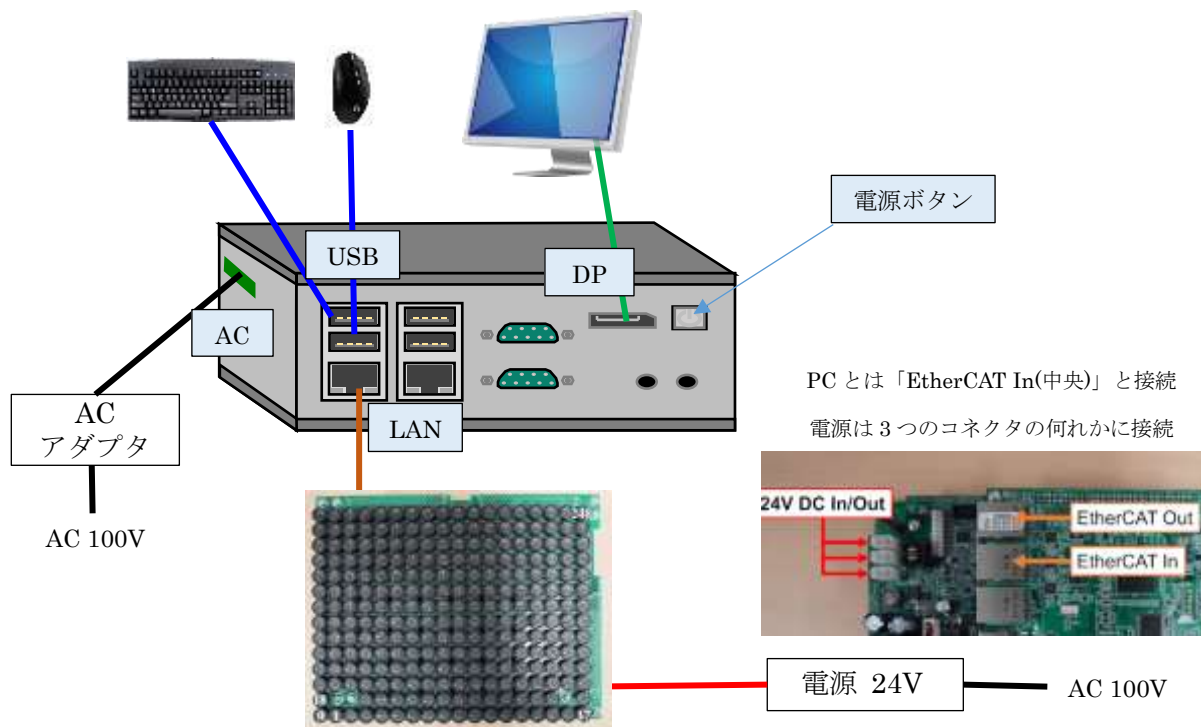
注意!

AUTD3 触覚ボードに目や耳を近づけないでください。

失明や耳を傷める恐れがあります。

2. ハードウェア接続

- AUTD3 触覚ボードキットの各機器を、下記の様に接続します。



- キーボード/マウス/モニタはキットに付属しませんので別途ご用意願います。
なお PC 本体の画像出力は DP(DisplayPort)となります。必要に応じて DP-HDMI 変換機器等もご用意下さい。
- PC 本体と AUTD3 ボードは、LAN ケーブル(カテゴリ 5e 以上)で接続します。
PC 側は LAN1(左側)に接続し、触覚ボード側は「中央のコネクタ」に接続します。
(上記画像の「EtherCAT in」に接続します)

3. 起動方法

- ・ PC 本体の電源ボタンを押して起動させます。
- ・ 起動後は下記の Ubuntu 画面がモニタに表示されます。



- ・ 出荷時はユーザログイン操作は省略させる設定となっています。
再ログイン等でユーザ/パスワードの入力を求められた場合は、下記を入力してください。
ユーザ名 : aiplay
パスワード : 0

上記は出荷時の初期パスワードとなります。
必要に応じてユーザ側で変更してください。

4. サンプルプログラム実行

4.1. サンプル一覧

- キットには下記の Python サンプルプログラムが格納されています。

タイトル	ファイル名	概要
単一焦点	sample01_single.py	空間の一点に触覚点を生成する
複数焦点	sample02_multi.py	空間に二点の触覚点を生成する
焦点移動及び ON/OFF	sample03_move.py	焦点を移動、及び、ON/OFF の切り替えを行う
ベッセルビーム	sample04_bessel.py	ベッセルビームによる線状に触覚点を生成する
STM(単焦点)	sample05_stm.py	STM(時空間変調)により動きのある触覚点を生成する

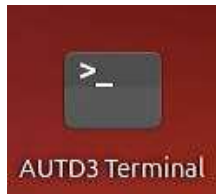
- サンプルプログラムはデスクトップの「PythonSample」フォルダ下に格納されています。

フォルダパスは下記となります。

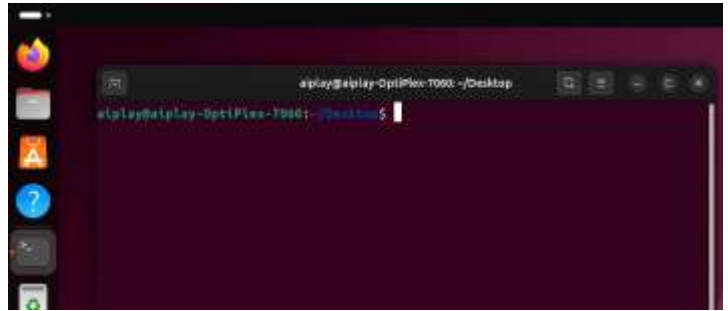
/home/aipay/Desktop/PythonSample

4.2. 実行方法

- サンプルプログラムの動作は、AUTD3 用の Python 仮想環境下で行う必要があります。
- デスクトップ上のアイコン「AUTD Terminal」を開く（ダブルクリック）ことにより、サンプルを実行できる環境のターミナル（コンソール画面）が表示されます。



ダブルクリック →



- Python プログラムの実行は下記コマンドにて行います。

python3 ファイル名

例) サンプル 1 (sample01_single.py) を実行する場合

python3 PythonSample/sample01_single.py

- 「AUTD3 Terminal」では、初期カレントディレクトリがデスクトップとなります。
この為、サンプルプログラムを実行する際は、
デスクトップ (/home/aiplay/Desktop) からの相対パス等で指定する必要があります。
- 標準のターミナル(デスクトップ右クリックメニューから「端末で開く」をクリック、あるいは、
Ctrl+Alt+T 等) でターミナルを開いた場合は、下記コマンドで実行環境に切り替える必要があります。

source ~/autd3env/bin/activate

なお標準ターミナルでは、カレントディレクトリがホーム (/home/aiplay) となっていますので、
サンプルプログラムのパス指定にはデスクトップを含める様にしてください。

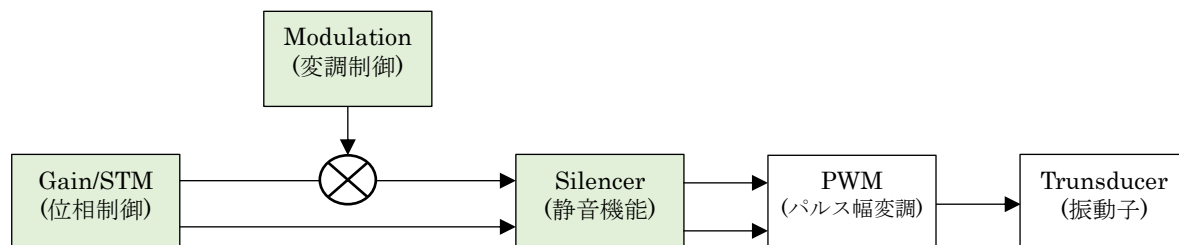
例) **python3 ~/Desktop/PythonSample/sample01_single.py**

5. サンプルプログラム解説

5.1. 共通

(1) AUTD3 触覚ボードの基本構造

- AUTD3 触覚ボード内の動作概念図を下記に示します。



- Gain : 位相と振幅を生成します。
 STM : 時間によって変化する位相と振幅を生成します。
 Modulation : AM 変調と加味します。
 Silencer : 消音化処理を行います。
 PWM : 各振動子を振動させる電気信号を生成します。
 Transducer : 電気信号を超音波に変換します。

位相制御にて位相と振幅を定義（触覚を発生させる位置等を決定）し、
 変調制御にて AM 変調付加（触覚の感じを強くするなどの加工）を加え、
 消音機能にて可聴音の除去（耳障りなノイズを抑制）させてから、
 パルス幅変調で最終的な信号を生成し振動子（超音波スピーカ）を動作させます。

- ユーザプログラムは緑色の部分に対してそれぞれの動作設定を指示することで、
 様々な触覚を生む音場を発生させることができます。

- Gain/STM 部 : どの位置にどのような焦点を発生させるかを指定します。
 Modulation 部 : 触覚を決定する波を指定します。
 Silencer 部 : 位相等が変化した場合に生じる可聴音を抑制する方法を指定します。

- AUTD3 触覚ボードを動作させるための基本的な流れは下記のようになります。

- ① デバイス構成定義
- ② 通信方法決定
- ③ デバイス接続
- ④ 消音処理定義と送信
- ⑤ 位相制御の定義と送信
- ⑥ 変調制御の定義と送信
- ⑦ 位置や状態の変化が生じるまで待機（変更する場合は、④⑤⑥を再度行う）
- ⑧ デバイス切断

サンプルでは主に①～③、⑤～⑥をまとめて記述する場合があります。

(2) 通信方法

- AUTD3 ライブラリでは複数の通信方法(Link)がサポートされています。

TwinCAT リンク

Beckhoff 社のソフトウェア製品を用いて通信を行います。(Windows 専用)

大量の AUTD3 ボードでも遅延を最小限にした高精度の通信を行えます。

EtherCrab リンク

Rust 言語で記述されたオープンソースなライブラリを使用して通信を行います。

数枚程度の AUTD3 ボード環境では最も安価で安定した通信を行えます。

Remote リンク

リモートサーバ経由で AUTD3 ボードと通信を行います。

プログラムの実行環境に通信用のライブラリ(TwinCAT 等)をインストールする必要がありません。

参考 : https://shinolab.github.io/autd3-doc/jp/Users_Manual/API/link.html

- キットに含まれるサンプルプログラムは Link(通信手段)として EtherCrab を使用しています。
- 通信を開始する際に何れかのリンクを指定する必要がありますが、基本的にそれ以降はリンク(通信方法)を気にすることなく、操作を行う事が可能です。
(EtherCrab を用いて動作確認後にリンク部分だけを TwinCAT 用に変更するだけで、それ以外のコードは修正の必要がありません)

5.2. 単一焦点サンプル(sample01_single.py)

(1) 動作概要

- このサンプルは、1 枚の AUTD3 デバイスの中央上部(デバイス表面から 15cm 上)に触覚を感じる焦点を発生させます。



- コンソールより下記コマンドを入力することで実行できます。

```
python3 PythonSample/sample01_single.py (🔴要確認: パス)
```

- Enter キーを押すと終了します。

サンプル解説 Python 編

(2) コード説明

- コード全体を下記に示します。

```
import numpy as np      # 科学技術用パッケージの準備
from pyautd3 import (  # AUTD3 パッケージの準備
    AUTD3,              # 触覚ボードの定義
    Controller,         # 触覚ボード制御
    Focus,              # 単一焦点を生成する Gain
    FocusOption,        # 単一焦点用オプション
    Silencer,           # 可聴音抑止
    Sine,               # 正弦波
    SineOption,         # 正弦波オプション
    Hz,                # 周波数指定
)
from pyautd3_link_ethercrab import EtherCrab, EtherCrabOption, Status # EtherCrab 用パッケージの準備

# デバイスエラー発生時の処理
def err_handler(idx: int, status: Status) -> None:
    # エラー内容の表示
    print( f"Device[{idx}]: {status}" )

# デモ開始
print( "<単一焦点デモ>" )

# デバイスへの接続(EtherCrab を使用)
autd = Controller.open(
    [ # AUTD3 ボードの指定
        AUTD3(pos=[0.0, 0.0, 0.0], rot=[1, 0, 0, 0]) # 1 枚目のボードの位置と向き
    ],
    EtherCrab( # EtherCrab での接続
        err_handler=err_handler, # エラー発生時の処理関数指定
        option=EtherCrabOption() # デフォルトオプション
    )
)

# 可聴音を抑えるための指示
autd.send( Silencer() )

# 動作指定
g = Focus( # 単一焦点の位相制御
    pos=autd.center() + np.array( [0.0, 0.0, 150.0] ), # 中心から上に 150mm の位置
    option=FocusOption(), # デフォルトオプション
)
m = Sine( # Sin 振幅
    freq=150 * Hz, # 150Hz
    option=SineOption(), # デフォルトオプション
)
print( "動作開始" )
autd.send( (m, g) ) # 位相制御と変調制御を指示

# キー入力待ち(終了指示待ち)
print( "Enter キー入力で終了" )
_ = input()

# デバイスへの接続終了
autd.close()
```

使用するパッケージの宣言

EtherCrab でエラーが発生した場合に
呼び出される関数を定義「1 枚ボード構成」と
「EtherCrab による接続」を定義・指定し
デバイスと接続(open)

消音化処理を有効にする指定を送信

単一焦点を生成する位相制御(Gain)を定義

正弦波による AM 変調を定義

位相制御(Gain)と変調制御を送信

Enter キーが押されるまで待機

デバイスから切断

- コードの詳細について説明します。

```
import numpy as np          # 科学技術用パッケージの準備
from pyautd3 import (      # AUTD3 パッケージの準備
    AUTD3,                  # 触覚ボードの定義
    Controller,             # 触覚ボード制御
    Focus,                  # 単一焦点を生成する Gain
    FocusOption,            # 単一焦点用オプション
    Silencer,               # 可聴音抑止
    Sine,                   # 正弦波
    SineOption,             # 正弦波オプション
    Hz,                     # 周波数指定
)
from pyautd3_link_ethcrab import EtherCrab, EtherCrabOption, Status # EtherCrab 用パッケージの準備
```

上記では使用するパッケージ及びクラスを宣言しています。

numpy : 科学学術用パッケージ。array 関数を使用する。
pyautd3 : AUTD3 デバイス制御パッケージ。import 句以下のクラスを使用する。
pyautd3_link_ethcrab : EtherCrab 用パッケージ。import 句以下のクラスを使用する。

```
# デバイスエラー発生時の処理
def err_handler(idx: int, status: Status) -> None:
    # エラー内容の表示
    print( f"Device[{idx}]: {status}" )
```

EtherCrab 接続を行う場合、エラー発生時に呼び出されるエラーハンドラ(関数)を指定する必要があります。
上記がその関数定義となります。

このサンプルのエラーハンドラ(err_handler)では、エラー内容をコンソールに表示しています。

デバイスへの接続(EtherCrab を使用)

```
autd = Controller.open(
    [ # AUTD3 ボードの指定
      AUTD3(pos=[0.0, 0.0, 0.0], rot=[1, 0, 0, 0]) # 1 枚目のボードの位置と向き
    ],
    EtherCrab( # EtherCrab での接続
      err_handler=err_handler, # エラー発生時の処理関数指定
      option=EtherCrabOption() # デフォルトオプション
    )
)
```

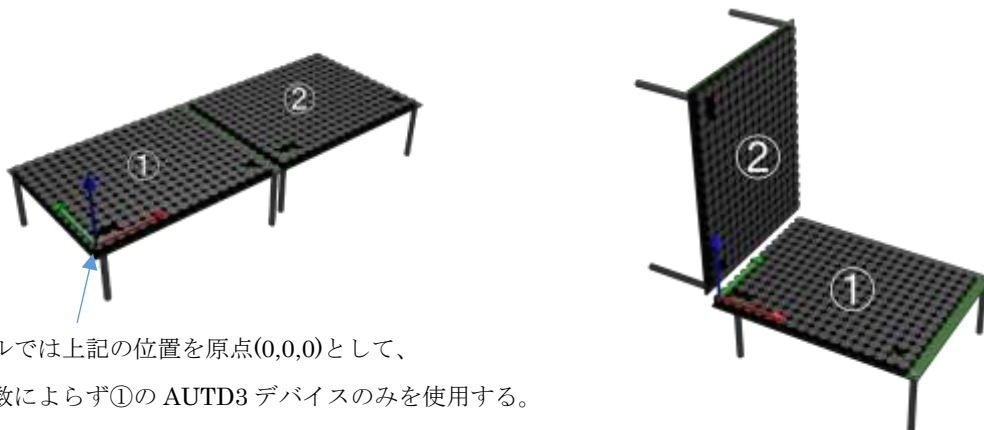
デバイスへの接続は `Controller` クラスの `open` メソッドにより行います。

引数として、AUTD3 デバイスのリストと接続に使用する Link として `EtherCrab` を指定しています。

本サンプルでは AUTD3 デバイスを 1 枚のみ使用しますのでリストは

基準位置 : (0, 0, 0) 原点
向き : (1, 0, 0, 0) 上向き

の 1 件のみとなっていますが、複数の AUTD3 デバイスを使用する場合は、各デバイス毎の位置と方向を列挙します。



サンプルでは上記の位置を原点(0,0,0)として、
接続台数によらず①の AUTD3 デバイスのみを使用する。

2 枚の AUTD3 デバイスを使用する場合は、
位置を横にずらした②の情報をリストに追加する。

上記の様な構成の場合、
向きを 90 度傾けた AUTD3 デバイスを定義する。

AUTD3 デバイスの配置情報を指定すれば、ライブラリが個々のデバイス(振動子)の振動を自動的に計算する。
(位相制御や変調制御では基本的に AUTD3 デバイスの位置や向きなどを考慮する必要がない。
なおデバイス毎や振動子毎にマスクしたり個別に指定することもできる)

`EtherCrab` を用いて接続を行う場合は、先に定義したエラーハンドラとオプションを指定します。
サンプルではデフォルトのオプション(引数なし)を指定していますが、
必要によってネットワークインターフェースや同期タイミング等を指定できます。

可聴音を抑えるための指示

```
autd.send( Silencer() )
```

上記では、可聴音を抑制する `Silencer` を指示しています。

「`Silencer().disable()`」を送信すれば、消音処理を無効化できます。

動作指定

```
g = Focus( # 単一焦点の位相制御
    pos=autd.center() + np.array( [0.0, 0.0, 150.0] ), # 中心から上に 150mm の位置
    option=FocusOption(), # デフォルトオプション
)
m = Sine( # Sin 振幅
    freq=150 * Hz, # 150Hz
    option=SineOption(), # デフォルトオプション
)
```

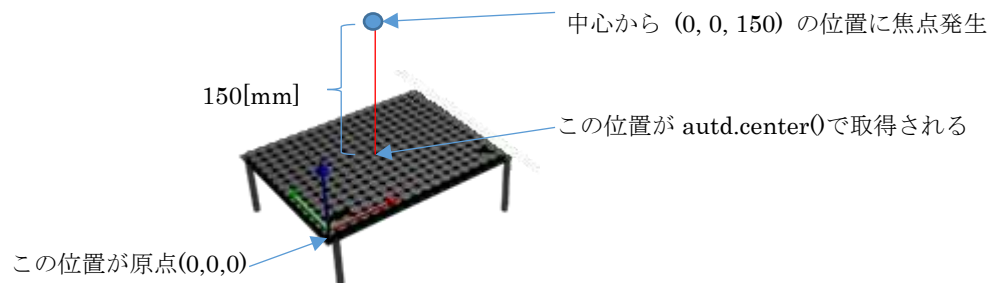
上記で「単一焦点の位相制御 `Focus`」と「正弦波変調 `Sine`」の定義を行っています。

`Focus` の引数は「焦点座標(`pos`)」と「オプション(`option`)」を指定します。

「`autd.center()`」は全 AUTD3 デバイスの中心位置を表し、

「`np.array( [0.0, 0.0, 150.0] )`」を加算することで

AUTD デバイス中心の上方向 150.0[mm]の位置を焦点座標としています。



変調制御として「正弦波:`Sine`」を指定しています。

変調なし(`Static`)の場合、動きがないと触覚を感じられません。

周波数は「150Hz」を指定していますが、この値を変更すると触覚が変化します。

サンプル解説 Python 編

```
print( "動作開始" )  
autd.send( (m, g) ) # 位相制御と変調制御を指示
```

```
# キー入力待ち(終了指示待ち)  
print( "Enter キー入力で終了" )  
_= input()
```

上記の「`send()`」にて位相制御と変調制御を AUTD3 デバイスに送信し、触覚を発生させます。

位相制御や変調制御は、送信する毎に上書きされます。(最後に送信された制御が使用されます)

Focus では単一位置で固定強度の焦点を発生させます。

細かく位相を変化させたい場合(焦点位置をブレさせたり、強さを変化させたい場合)などは
「STM：時空間変調」が有効です。

送信後は Enter キーが押されるまで待機します。

(プログラムが終了すると超音波出力も終了するため、
キー入力があるまで出力が継続される様にしています)

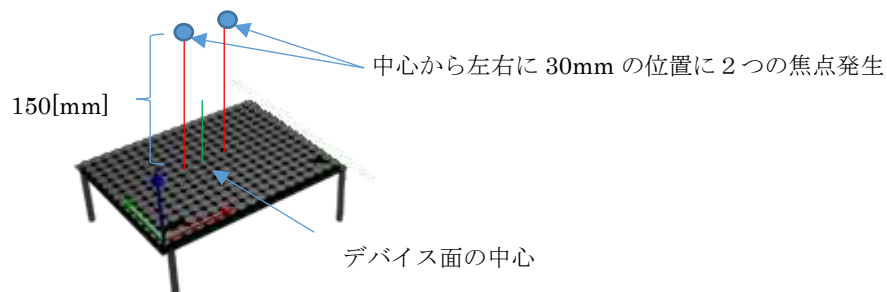
```
# デバイスへの接続終了  
autd.close()
```

上記の「`close()`」により、AUTD デバイスとの接続を終了します。

5.3. 複数焦点サンプル(sample02_multi.py)

(1) 動作概要

- このサンプルは、1 枚の AUTD3 デバイスの上に 2 つの触覚を感じる焦点を発生させます。



- 多焦点生成アルゴリズムとして「GSPAT:Gershberg-Saxon for Phased Arrays of Transducers」を使用しています。
- コンソールより下記コマンドを入力することで実行できます。

```
python3 PythonSample/sample02_multi.py
```
- Enter キーを押すと終了します。

サンプル解説 Python 編

(2) コード説明

- コード全体を下記に示します。

```
import numpy as np      # 科学技術用パッケージの準備

from pyautd3 import (   # AUTD3パッケージの準備
    AUTD3,              # 触覚ボードの定義
    Controller,         # 触覚ボード制御
    Silencer,           # 可聴音抑止
    Sine,               # 正弦波
    SineOption,         # 正弦波オプション
    Hz,                # 周波数指定
)

from pyautd3_link_ethercrab import EtherCrab, EtherCrabOption, Status # EtherCrab用パッケージの準備
from pyautd3.gain.holo import ( # 多焦点音場関連のパッケージ準備
    GSPAT,              # GSPAT方式
    GSPATOption,        # GSPAT方式オプション
    Pa                  # 圧力指定
)

# デバイスエラー発生時の処理
def err_handler(idx: int, status: Status) -> None:
    # エラー内容の表示
    print( f"Device[{idx}]: {status}" )

# デモ開始
print( "<複数焦点デモ>" )

# デバイスへの接続(EtherCrabを使用)
autd = Controller.open(
    [ # AUTD3ボードの指定
        AUTD3(pos=[0.0, 0.0, 0.0], rot=[1, 0, 0, 0]) # 1枚目のボードの位置と向き
    ],
    EtherCrab( # EtherCrabでの接続
        err_handler=err_handler, # エラー発生時の処理関数指定
        option=EtherCrabOption() # デフォルトオプション
    )
)

# 可聴音を抑えるための指示
autd.send( Silencer() )

# 動作指定
center = autd.center() + np.array( [0.0, 0.0, 150.0]) # 焦点の中心:中心から上に150mmの位置
g = GSPAT(
    foci=[ (center - np.array([30.0, 0.0, 0.0]), 10e3 * Pa), # 焦点1:中心から横に+30.0mm
           (center + np.array([30.0, 0.0, 0.0]), 10e3 * Pa)], # 焦点2:中心から横に-30.0mm
    option=GSPATOption(),
)

m = Sine( # Sin振幅
    freq=150 * Hz, # 150Hz
    option=SineOption(), # デフォルトオプション
)

print( "動作開始" )
autd.send( (m, g) ) # 位相制御と変調制御を指示

# キー入力待ち(終了指示待ち)
print( "Enterキー入力で終了" )
_ = input()

# デバイスへの接続終了
autd.close()
```

使用するパッケージの宣言

EtherCrab でエラーが発生した場合に
呼び出される関数を定義

「1枚ボード構成」と
「EtherCrab による接続」を定義・指定し
デバイスと接続(open)

消音化処理を有効にする指定を送信

2つの焦点を生成する位相制御(Gain)を定義

正弦波による AM 変調を定義

位相制御(Gain)と変調制御を送信

Enter キーが押されるまで待機

デバイスから切断

サンプル解説 Python 編

- コードの詳細について説明します。

なお可聴音を抑える為の指示部分までは、「単一焦点サンプル」とほぼ同様のため

「5.2 単一焦点サンプル」を参照願います。

ただし、下記のパッケージ宣言が追加されています。

```
from pyautd3.gain.holo import ( # 多焦点音場関連のパッケージ準備
    GSPAT,                      # GSPAT方式
    GSPATOption,               # GSPAT方式オプション
    Pa                          # 圧力指定
)
```

上記は、本サンプルで使用している多焦点生成アルゴリズム

「GSPAT:Gershberg-Saxon for Phased Arrays of Transducers」を使用するための宣言となります。

動作指定

```
center = autd.center() + np.array([0.0, 0.0, 150.0]) # 焦点の中心:中心から上に150mmの位置
g = GSPAT(
    foci=[(center - np.array([30.0, 0.0, 0.0]), 10e3 * Pa), # 焦点1:中心から横に+30.0mm
          (center + np.array([30.0, 0.0, 0.0]), 10e3 * Pa)], # 焦点2:中心から横に-30.0mm
    option=GSPATOption(),
)
```

上記で、GSPAT アルゴリズムによる 2 つの焦点を生成する位相制御を定義しています。

変数「center」には AUTD3 デバイスの中心から上 150mm の座標=2 焦点の中心が格納されます。

GSPAT の引数は、「焦点座標リスト」と「オプション」を指定します。

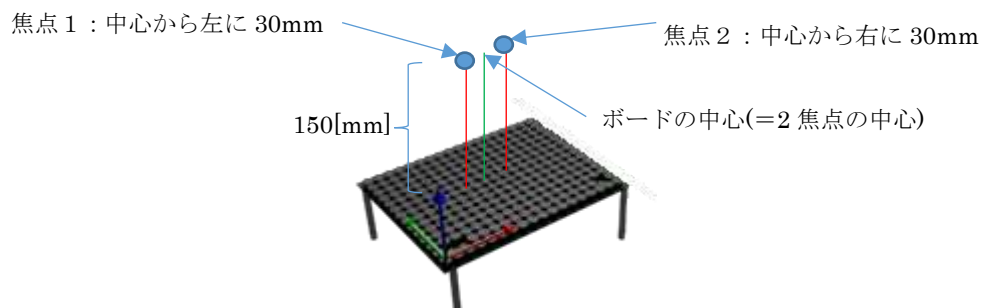
「焦点座標リスト」は「焦点座標」と「圧力強度」からなるタプルのリストで、

上記コードの場合、

2 焦点の中心(center) から 左に 30mm の位置で、圧力 10×10^3 [Pa]

2 焦点の中心(center) から 右に 30mm の位置で、圧力 10×10^3 [Pa]

の 2 要素が指定されています。



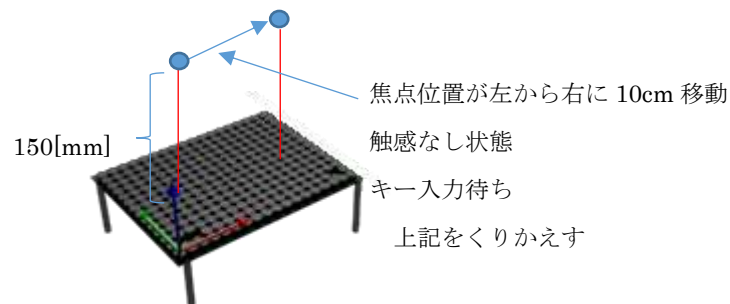
変調制御の定義(Sin 振幅)、及び、位相制御と変調制御の送信と待機部分についても

「単一焦点サンプル」と同様となります。

5.4. 焦点移動及び ON/OFF サンプル(sample03_move.py)

(1) 動作概要

- このサンプルは、AUTD3 デバイスの上の触覚を感じる焦点が左右に移動し、右端に達した時に触感が一定期間(1 秒)停止します。



- 焦点の移動は、ミリ単位で位置をずらして単一焦点を発生させています。
- 触感出力の停止は、Null という Gain を指示(送信)する事で実現します。

Focus 等の Gain を送信 : 触感 ON

Null を送信 : 触感 OFF

- コンソールより下記コマンドを入力することで実行できます。

```
python3 PythonSample/sample03_move.py
```

- 「空 Enter 入力で繰り返し(なにか入力して Enter で終了)」が表示された時、何も入力せずに Enter キーを押すと繰り返し実行します。
何か入力して Enter キーを押すと、終了します。

AUTD3 触覚ボードキット

サンプル解説 Python 編

(2) コード説明

- コード全体を下記に示します。

```
import numpy as np # 科学技術用パッケージの準備
```

```
from pyautd3 import ( # AUTD3パッケージの準備
    AUTD3, # 触覚ボードの定義
    Controller, # 触覚ボード制御
    Focus, # 単一焦点を生成するGain
    FocusOption, # 単一焦点用オプション
    Null, # 出力停止Gain
    Silencer, # 可聴音抑止
    Sine, # 正弦波
    SineOption, # 正弦波オプション
    Hz, # 周波数指定
)
```

```
from pyautd3_link_ethercrab import EtherCrab, EtherCrabOption, Status # EtherCrab用パ
import time # 時間調整用のパッケージ準備
```

```
# デバイスエラー発生時の処理
def err_handler(idx: int, status: Status) -> None:
    # エラー内容の表示
    print( f"Device[{idx}]: {status}" )
```

```
# デモ開始
print( "＜焦点移動デモ＞" )
```

```
# デバイスへの接続(EtherCrabを使用)
autd = Controller.open(
    [ # AUTD3ボードの指定
        AUTD3(pos=[0.0, 0.0, 0.0], rot=[1, 0, 0, 0]) # 1枚目のボードの位置と向き
    ],
    EtherCrab( # EtherCrabでの接続
        err_handler=err_handler, # エラー発生時の処理関数指定
        option=EtherCrabOption() # デフォルトオプション
    )
)
```

```
# 可聴音を抑えるための指示
autd.send( Silencer() )
```

```
# 正弦波用意(移動中に変更しないため事前に指示)
m = Sine( # Sin振幅
    freq=150 * Hz, # 150Hz
    option=SineOption() ) # デフォルトオプション
autd.send( m ); # 変調制御を指示
```

```
# 入力があるまでループ
print( "動作開始" )
loop = True
while loop:
```

```
    # -50mmから+50mmまで移動
    for idx in range(-50, 50):
```

```
        # 単一焦点設定
        g = Focus(
            pos=autd.center() + np.array( [idx, 0.0, 150.0] ), # 中心から横に-50～+50mmの位置を指定
            option=FocusOption() ) # デフォルトオプション
        autd.send( g ) # 位相制御を指示
```

```
        # 時間調整
        time.sleep( 0.01 ) # 0.01秒(10ミリ秒)待機
```

```
# 停止
autd.send( Null() )
```

```
# 終了確認
ans = input( "空Enter入力で繰り返し(なにか入力してEnterで終了)" )
if ans != "":
    break # 空入力でない場合、終了
```

```
# デバイスへの接続終了
autd.close()
```

使用するパッケージの宣言

EtherCrab でエラーが発生した場合に
呼び出される関数を定義

「1枚ボード構成」と
「EtherCrab による接続」を定義・指定し
デバイスと接続(open)

消音化処理を有効にする指定を送信

正弦波による AM 変調の定義と指示

移動ループ
(焦点位置を中心から-50mm～+50mm)

位相制御(Gain)の定義と送信

時間調整の為の待機

出力停止指示

終了確認

デバイスから切断

サンプル解説 Python 編

- コードの詳細について説明します。

可聴音を抑える為の指示部分までと最後の切断処理は、「単一焦点サンプル」とほぼ同様のため「5.2 単一焦点サンプル」を参照願います。

ただし、下記のパッケージ宣言が追加されています。

```
import time                # 時間調整用のパッケージ準備
```

上記は、時間調整(待機)する為の関数(sleep)を使用するための宣言となります。

```
# 正弦波用意(移動中に変更しないため事前に指示)
m = Sine( # Sin振幅
    freq=150 * Hz, # 150Hz
    option=SineOption() ) # デフォルトオプション
autd.send( m ); # 変調制御を指示
```

上記で正弦波による AM 変調を定義し、送信しています。

AM 変調自体は単焦点サンプルと同じですが、焦点が移動中は変化しないため、ループの前で送信しています。

```
# 入力があるまでループ
print( "動作開始" )
loop = True
while loop:

    # -50mmから+50mmまで移動
    for idx in range(-50, 50):

        # 単一焦点設定
        (略)

        # 時間調整

    # 停止
    (略)

    # 終了確認
    (略)
```

上記は繰り返し動作（左から右への焦点移動、停止、終了確認）のループ処理になります。
(各項目のコードは次ページ以降で解説するため省略しています)

終了確認で変数 loop が false になるまで繰り返します。

```
# 単一焦点設定
g = Focus(
    pos=autd.center() + np.array( [idx, 0.0, 150.0] ), # 中心から横に-50～+50mmの位置を指定
    option=FocusOption() ) # デフォルトオプション
autd.send( g ) # 位相制御を指示
```

ループ内の単一焦点設定では、移動する焦点の位置に合わせた Gain(Focus)を生成し、送信しています。
autd.center()が全体の中心位置であり、-50～50 の範囲で変化する変数 idx を x 座標オフセットとして
焦点位置 pos を設定しています。

```
# 時間調整
time.sleep( 0.01 ) # 0.01秒(10ミリ秒)待機
```

上記の時間調整は、ゆっくりと焦点位置を移動させるための待機処理となります。

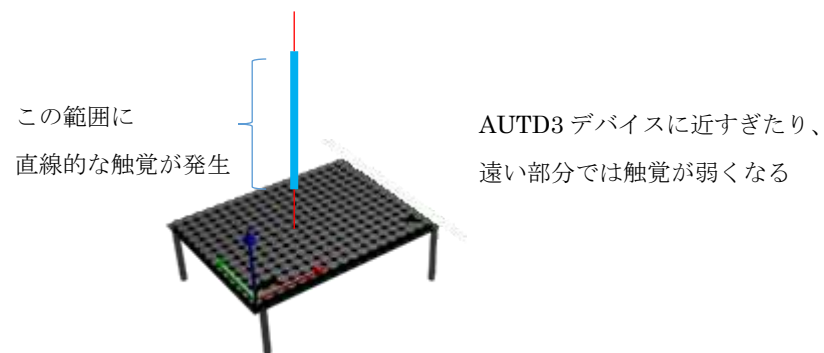
```
# 停止
autd.send( Null() )
```

横移動ループが終了後、上記にて出力を停止しています。
ループして次の単一焦点設定を送信した時に出力は(新しい位置で)再開されます。

5.5. ベッセルビームサンプル(sample04_bessel.py)

(1) 動作概要

- このサンプルは、AUTD3 デバイスにてベッセルビームを発生させ、一か所の焦点ではなく直線的な範囲の触覚を発生させます。



- コンソールより下記コマンドを入力することで実行できます。

```
python3 PythonSample/sample04_bessel.py
```

- Enter キーを押すと、終了します。

(2) コード説明

- コード全体を下記に示します。

```
import numpy as np # 科学技術用パッケージの準備

from pyautd3 import ( # AUTD3パッケージの準備
    AUTD3, # 触覚ボードの定義
    Controller, # 触覚ボード制御
    Bessel, # ベッセルビームを生成するGain
    BesselOption, # ベッセルビーム用オプション
    rad, # ラジアン単位指定
    Silencer, # 可聴音抑止
    Sine, # 正弦波
    SineOption, # 正弦波オプション
    Hz, # 周波数指定
)

from pyautd3_link_ethercrab import EtherCrab, EtherCrabOption, Status # EtherCrab用パッケージの準備

# デバイスエラー発生時の処理
def err_handler(idx: int, status: Status) -> None:
    # エラー内容の表示
    print( f"Device[{idx}]: {status}" )

# デモ開始
print( "<ベッセルビームデモ>" )

# デバイスへの接続(EtherCrabを使用)
autd = Controller.open(
    [ # AUTD3ボードの指定
        AUTD3(pos=[0.0, 0.0, 0.0], rot=[1, 0, 0, 0]) # 1枚目のボードの位置と向き
    ],
    EtherCrab( # EtherCrabでの接続
        err_handler=err_handler, # エラー発生時の処理関数指定
        option=EtherCrabOption() # デフォルトオプション
    )
)

# 可聴音を抑えるための指示
autd.send( Silencer() )

# 動作指定
g = Bessel( # ベッセルビームの位相制御
    apex=autd.center(),
    direction=np.array([0.0, 0.0, 1.0]),
    theta=13.0 / 180 * np.pi * rad,
    option=BesselOption()
)

m = Sine( # Sin振幅
    freq=150 * Hz, # 150Hz
    option=SineOption(), # デフォルトオプション
)

print( "動作開始" )
autd.send( (m, g) ) # 位相制御と振幅制御を指示

# キー入力待ち(終了指示待ち)
print( "Enterキー入力で終了" )
_ = input()

# デバイスへの接続終了
autd.close()
```

使用するパッケージの宣言

EtherCrab でエラーが発生した場合に
呼び出される関数を定義「1枚ボード構成」と
「EtherCrabによる接続」を定義・指定し
デバイスと接続(open)

消音化処理を有効にする指定を送信

ベッセルビームを生成する
位相制御(Gain)を定義

正弦波による AM 変調を定義

位相制御(Gain)と変調制御を送信

Enter キーが押されるまで待機

デバイスから切断

- コードの詳細について説明します。

可聴音を抑える為の指示部分までと最後の待機/切断処理は、「単一焦点サンプル」とほぼ同様のため「5.2 単一焦点サンプル」を参照願います。

ただし、下記のパッケージ宣言が追加されています。

```
Bessel,          # ベッセルビームを生成するGain
BesselOption,    # ベッセルビーム用オプション
rad,             # ラジアン単位指定
```

上記は、ベッセルビームを生成する位相制御(Bessel)を使用する為の宣言となります。

動作指定

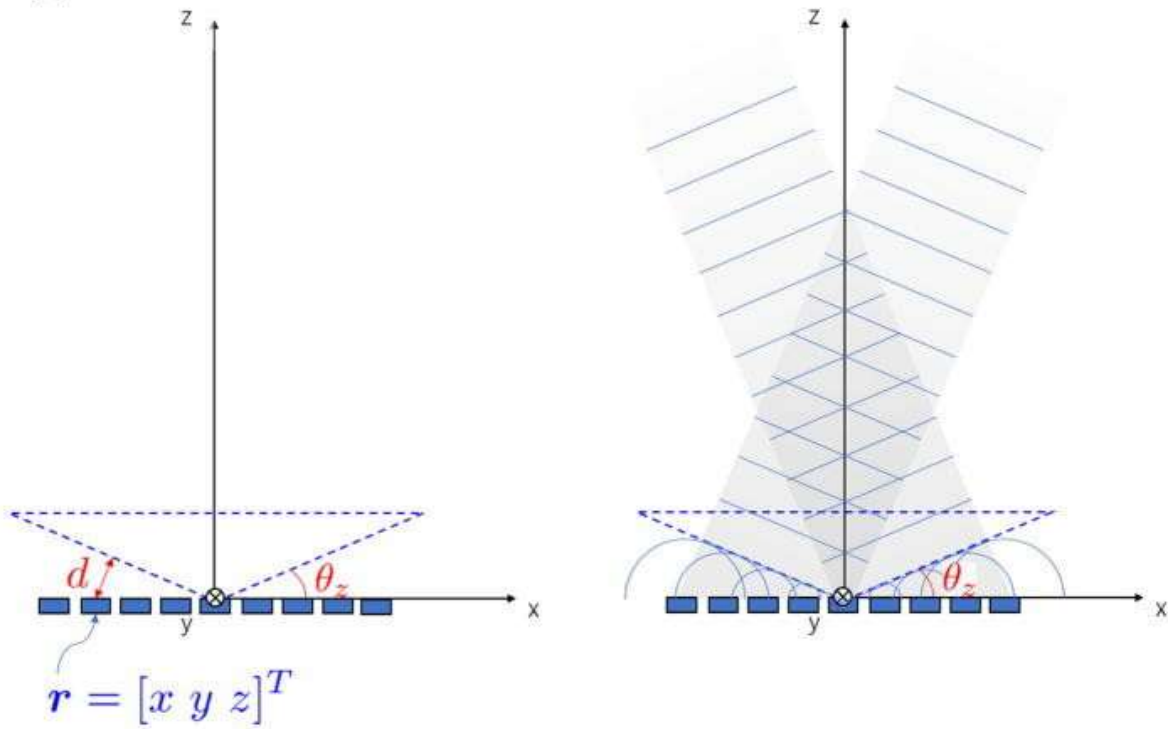
```
g = Bessel( # ベッセルビームの位相制御
    apex=autd.center(),
    direction=np.array([0.0, 0.0, 1.0]),
    theta=13.0 / 180 * np.pi * rad,
    option=BesselOption()
)
m = Sine( # Sin振幅
    freq=150 * Hz, # 150Hz
    option=SineOption(), # デフォルトオプション
)
print("動作開始")
autd.send( (m, g) ) # 位相制御と振幅制御を指示
```

上記でベッセルビームを発生させる位相制御と AM 変調制御を定義し送信しています。

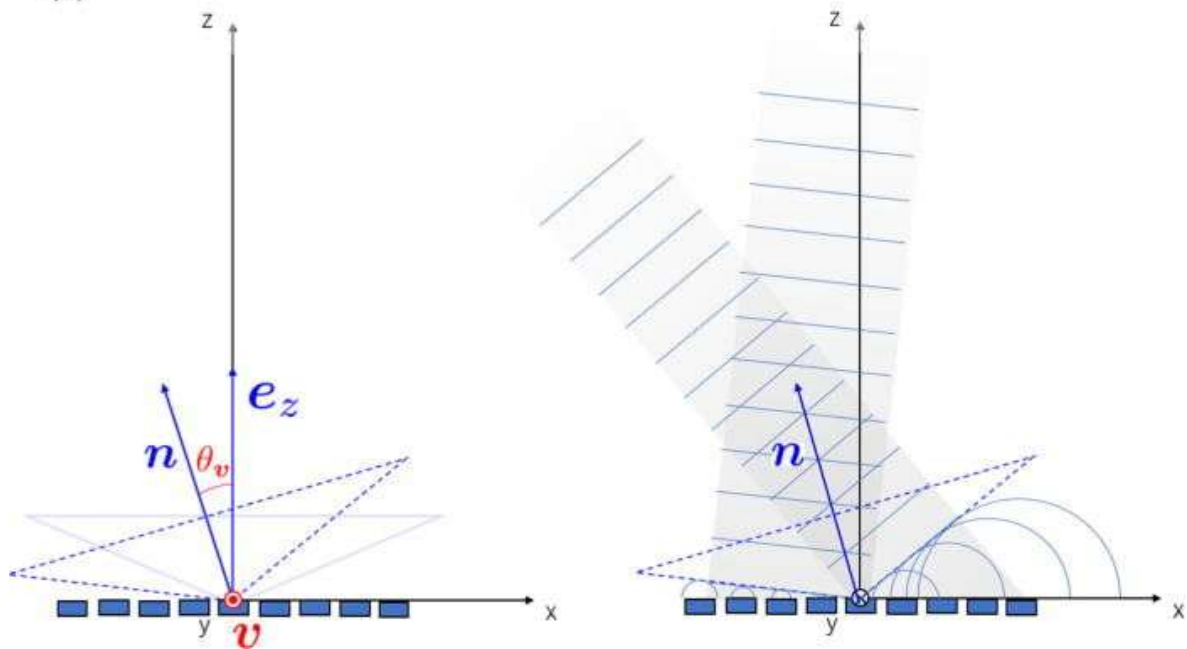
ベッセルビームの位相制御は、ビームを生成する仮想円錐の頂点(apex)とビームの方向(direction)、及び、ビームに垂直な面と仮想円錐側面との角度(theta)を指定して定義します。(次ページの図を参照)

変調制御は単焦点サンプルと同じ「正弦波:Sine」を指定しています。

(a)



(b)

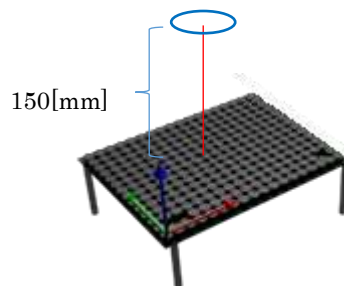


参考 : https://shinolab.github.io/autd3-doc/jp/Users_Manual/API/gain/bessel.html

5.6. STM サンプル(sample05_stm.py)

(1) 動作概要

- このサンプルは、Spatio-Temporal Modulation:時空間変調をもちいて、時間とともに変化する焦点を発生させます。



中心から上 150mm の位置を中心に
半径 30mm の円を描く様に触覚が発生
(1 秒周期で回転)

- コンソールより下記コマンドを入力することで実行できます。

```
python3 PythonSample/sample05_stm.py
```

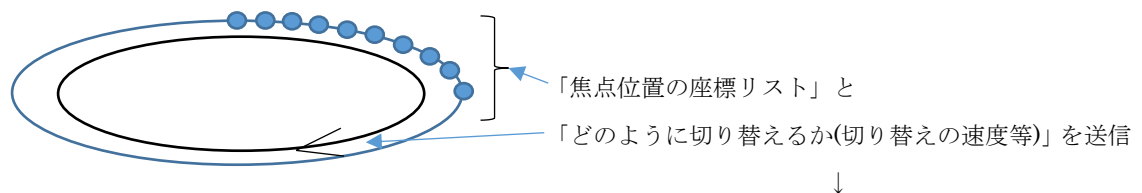
- Enter キーを押すと、終了します。
- サンプルは単焦点サンプルと同様に位相制御等を送信後はキー入力があるまでプログラムは待機状態となりますが、焦点移動サンプルの様に触覚の移動が繰り返されます。
- ライブラリでは下記の STM 機能を提供しています。

FociSTM : 8 つまでの焦点を制御する STM

GainSTM : 任意の Gain を組み合わせて制御する STM

本サンプルでは FociSTM を使用します。

- FociSTM では、「焦点の発生座標リスト」と「焦点の切り替え周期」を決定するパラメータを AUTD3 デバイスに送信し、AUTD3 デバイス内で自動的に焦点座標を切り替えさせることで焦点の動きを実現します。



AUTD3 デバイスが、自動的に焦点位置を切り替える

本サンプルでは、「200 個の焦点座標リスト」と「1Hz」のパラメータを送信していますので、円を描く 200 点の焦点が 1 秒周期で回転するような触覚が発生します。

(2) コード説明

- コード全体を下記に示します。

```
import numpy as np # 科学技術用パッケージの準備

from pyautd3 import ( # AUTD3パッケージの準備
    AUTD3, # 触覚ボードの定義
    Controller, # 触覚ボード制御
    FociSTM, # Focus制御STM
    Silencer, # 可聴音抑止
    Static, #
    Hz, # 周波数指定
)
from pyautd3_link_ethercrab import EtherCrab, EtherCrabOption, Status # EtherCrab用パッケージの準備

# デバイスエラー発生時の処理
def err_handler(idx: int, status: Status) -> None:
    # エラー内容の表示
    print( f"Device[{idx}]: {status}" )

# デモ開始
print( "<時空間変調デモ>" )

# デバイスへの接続(EtherCrabを使用)
autd = Controller.open(
    [ # AUTD3ボードの指定
        AUTD3(pos=[0.0, 0.0, 0.0], rot=[1, 0, 0, 0]) # 1枚目のボードの位置と向き
    ],
    EtherCrab( # EtherCrabでの接続
        err_handler=err_handler, # エラー発生時の処理関数指定
        option=EtherCrabOption() # デフォルトオプション
    )
)

# 可聴音を抑えるための指示
autd.send( Silencer() )

# 動作指定
radius = 30.0 # 動きの半径
reso = 200 # 動きの分解能
center = autd.center() + np.array([0.0, 0.0, 150.0]) # 動きの中心位置
stm = FociSTM(
    foci=(center + radius * np.array([np.cos(theta), np.sin(theta), 0]))
    for theta in ( 2.0 * np.pi * i / reso for i in range(reso) ) # 分解能に合わせて角度決定
),
    config=1.0 * Hz,
)
m = Static()
print( "動作開始" )
autd.send( (m, stm) ) # 位相制御と振幅制御を指示

# キー入力待ち(終了指示待ち)
print( "Enterキー入力で終了" )
_= input()

# デバイスへの接続終了
autd.close()
```

使用するパッケージの宣言

EtherCrab でエラーが発生した場合に
呼び出される関数を定義「1枚ボード構成」と
「EtherCrab による接続」を定義・指定し
デバイスと接続(open)

消音化処理を有効にする指定を送信

焦点座標のリストと周期を指定して
FociSTM を定義

固定の変調制御を定義

位相制御(STM)と変調制御を送信

Enter キーが押されるまで待機

デバイスから切断

サンプル解説 Python 編

- コードの詳細について説明します。

可聴音を抑える為の指示部分までと最後の待機/切断処理は、「単一焦点サンプル」とほぼ同様のため「5.2 単一焦点サンプル」を参照願います。

ただし、下記のパッケージ宣言が追加されています。

```
FociSTM,          # Focus制御STM
Static,           #
Hz,               # 周波数指定
```

上記は、FociSTM による STM と固定の変調制御を使用する為の宣言となります。

```
# 動作指定
radius = 30.0    # 動きの半径
reso = 200       # 動きの分解能
center = autd.center() + np.array([0.0, 0.0, 150.0]) # 動きの中心位置
stm = FociSTM(
    foci=(center + radius * np.array([np.cos(theta), np.sin(theta), 0]))
        for theta in (2.0 * np.pi * i / reso for i in range(reso)) # 分解能に合わせて角度決定
    ),
    config=1.0 * Hz,
)
```

上記では中心位置(autd.center())を中心とした半径:radius(30.0)[mm]の円の円周上の200点の座標リスト(foci)と1Hzで繰り返す周期設定(config)を指定し、FociSTM(stm)を定義しています。

```
m = Static()
```

本サンプルでは変調制御として Static を使用しています。

他サンプルの正弦波 Sine と違い AM 変調が付与されませんが、STM により焦点が次々と変化するため触感を感じられます。

6. その他

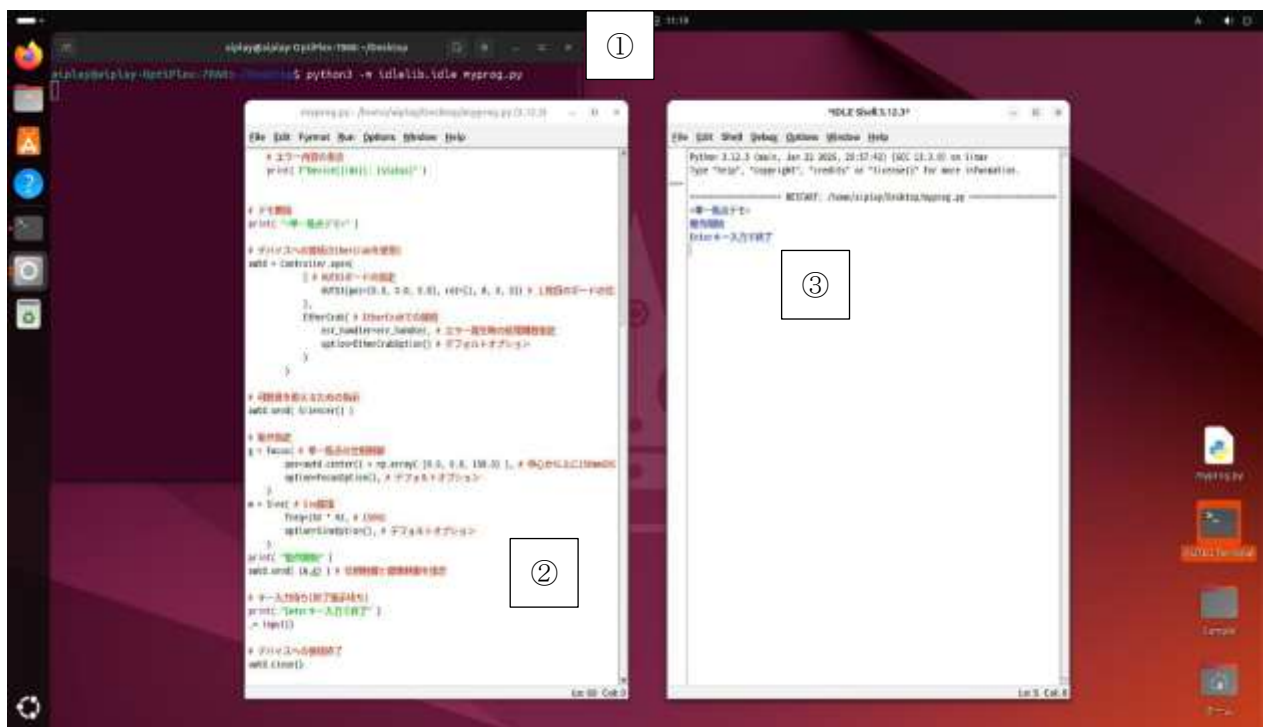
6.1. 独自プログラムの作成と実行

- Ubuntu 標準のテキストエディタや Python 付属の総合開発環境 IDLE がインストールされています。これらを使用してプログラムを編集/保存し、サンプルプログラムと同様のコマンドで実行することができます。
(保存先を/home/aipplay/Desktop 下にする事でデスクトップにアイコンが表示され、サンプルと同様の手順で実行できます)

- 「AUTD3 Terminal」にて下記コマンドを実行すれば
総合開発環境 IDLE 下でプログラムの編集と実行が可能です。

`python3 -m idlelib.idle` ファイル名

IDLE を使用した独自プログラムの作成/実行例



- ① 「AUTD3 Terminal」上で上記コマンドを実行
- ② 編集ウィンドウ内にプログラムを入力
- ③ メニュー「Run」-「Run Module」(F5)で実行ウィンドウが表示されプログラムが実行

6.2. 実行時のエラーなど

- サンプル実行時に下記の様な出力となった場合、

```

aisplay@aisplay-OptiPlex-7060:~/Desktop$ python3 Sample/sample01_single.py
<単一焦点デモ>

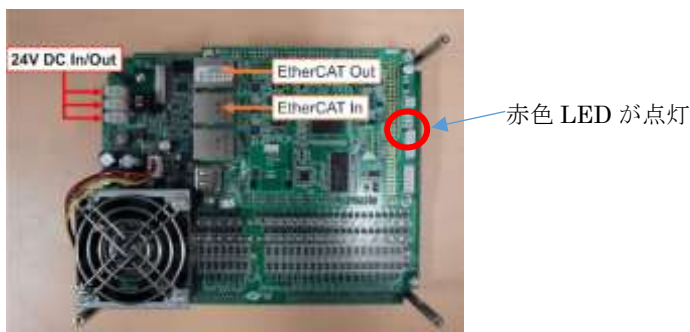
thread 'tokio-runtime-worker' (5040) panicked at /home/runner/.cargo/registry/src/index.crates.io-1949cf8c6b5b557f/ethcrab-0.6.0/src/std/unix/mod.rs:178:9:
index out of bounds: the len is 16 but the index is 16
note: run with 'RUST_BACKTRACE=1' environment variable to display a backtrace
Traceback (most recent call last):
  File "/home/aisplay/Desktop/Sample/sample01_single.py", line 29, in <module>
    autd = Controller.open()
            ~~~~~
  File "/home/aisplay/autd3env/lib/python3.12/site-packages/pyautd3/controller/controller.py", line 125, in open
    return Controller.open_with_option(devices, link, SenderOption())
            ~~~~~
  File "/home/aisplay/autd3env/lib/python3.12/site-packages/pyautd3/controller/controller.py", line 137, in open_with_option
    ptr = _validate_ptr()
            ~~~~~
  File "/home/aisplay/autd3env/lib/python3.12/site-packages/pyautd3/native_methods/utils.py", line 63, in _validate_ptr
    raise AUTDError(err)
pyautd3.autd_error.AUTDError: No AUTD3 device found
aisplay@aisplay-OptiPlex-7060:~/Desktop$

```

pyautd3.autd_error.AUTDError: No AUTD3 device found

下記の点を確認してください。

- AUTD3 デバイスの電源ケーブルが正しく接続されていることを確認してください。
また電源ユニット(24V)の電源が ON になっていることを確認してください。



AUTD3 デバイスは電源が入っている場合、上記の LED が点灯します。

- AUTD3 デバイスと PC が正しいコネクタに接続されていることを確認して下さい。
AUTD3 デバイス側は中央の LAN コネクタに接続します。
正しく接続されている場合、LAN コネクタの緑と黄色の LED が点灯します。
(PC とは HUB 等を経由せずに接続してください。また別のケーブルでの接続を確認下さい)
- WiFi デバイスや USB-LAN デバイス(有線 LAN)等は接続しないでください。
複数のネットワークデバイスがあった場合でも、
通常はライブラリが自動的に適切なネットワークデバイスを選択しますが、
失敗した場合上記の様な状況になる場合があります。

- ```
aiplay@aiplay-OptiPlex-7060: ~/Desktop
aiplay@aiplay-OptiPlex-7060:~/Desktop$ python3 Sample/sample01_single.py
Traceback (most recent call last):
 File "/home/aiplay/Desktop/Sample/sample01_single.py", line 1, in <module>
 import numpy as np # 科学技術用パッケージの準備
 ^^^^^^^^^^^^^^^^^^^^^^^^^
ModuleNotFoundError: No module named 'numpy'
aiplay@aiplay-OptiPlex-7060:~/Desktop$
```

デスクトップの「AUTD3 Terminal」アイコンから起動するターミナルから入力するか、下記コマンドで仮想環境に切り替えた後にプログラムを実行して下さい。

- AUTD3 デバイスを長時間動作させて熱を持った場合、自動的に冷却用のファンが動き出します。ファンが動作している状態でも動作に問題はありません。

30 / 30